

Dining Philosophers

OS Project – Baseline vs Monitor Solution | Python

1. Baseline Approach – Why It Can Deadlock

Each philosopher grabs the **left fork** first, then tries for the **right fork**. This works fine most of the time, but if all 5 philosophers grab their left fork at exactly the same moment, each one then waits for a right fork that is held by their neighbour. Nobody can move forward — this is a **deadlock**.

One-sentence deadlock scenario: All five philosophers pick up their left fork at the same time, then each waits for a right fork that nobody will ever release.

Key code (baseline.py):

```
forks[left].acquire() # grab left fork
forks[right].acquire() # try right fork -- may block forever if neighbour also waiting
```

2. Monitor Solution – How It Works

The monitor uses a shared **state[]** array (THINKING / HUNGRY / EATING) and one **condition variable per philosopher** to coordinate access safely.

test(i) — checks if philosopher i can eat:

Only allows eating if: left neighbour is NOT eating, i is HUNGRY, and right neighbour is NOT eating. All three must be true at once. This makes sure two neighbours never eat at the same time.

pickup(i) — philosopher wants to eat:

Sets state to HUNGRY, calls test(i). If still not EATING, calls wait() — this releases the lock and puts the thread to sleep until a neighbour wakes it up.

putdown(i) — philosopher finishes eating:

Sets state back to THINKING, then calls test() on both neighbours. This gives them a chance to start eating, so nobody starves.

Key code (monitor.py):

```
def test(self, i):
    if (self.state[left] != EATING and
        self.state[i] == HUNGRY and
        self.state[right] != EATING):
        self.state[i] = EATING
        self.cond[i].notify()

def pickup(self, i):
    with self.lock:
        self.state[i] = HUNGRY
        self.test(i)
        while self.state[i] != EATING:
            self.cond[i].wait() # sleep until woken up

def putdown(self, i):
    with self.lock:
        self.state[i] = THINKING
        self.test(left) # wake up left neighbour if possible
        self.test(right) # wake up right neighbour if possible
```

3. Sample Output

Part A – Baseline:

```
PART A: BASELINE (may deadlock)
```

Run time: 15.0s

Philosopher	Eat Count	Max Wait (s)
P0	143	0.0294
P1	146	0.0275
P2	147	0.0315
P3	147	0.0294
P4	144	0.0293

>>> DEADLOCK detected

Part B – Monitor:

PART B: MONITOR SOLUTION (correct)
Run time: 2.0s

Philosopher	Eat Count	Max Wait (s)
P0	100	0.0166
P1	100	0.0144
P2	100	0.0099
P3	100	0.0123
P4	100	0.0141

>>> No deadlock. All philosophers ate >= 3 times. PASS.

4. Exam Question Answers

Q1: What condition lets philosopher i enter EATING in test(i)?

Left neighbour is not eating, i is HUNGRY, and right neighbour is not eating — all three must be true.

Q2: Why does pickup(i) call wait() sometimes?

If a neighbour is already eating, i cannot eat yet. wait() puts the thread to sleep and releases the lock so others can still use the monitor.

Q3: Why does putdown(i) call test() on both neighbours?

When a philosopher stops eating, one or both neighbours might now be able to eat. Calling test() checks this and wakes them up — preventing starvation.

Q4: One deadlock scenario in the baseline:

All five philosophers grab their left fork at the same time, then each waits forever for a right fork their neighbour is holding.